Images entières en parallèle

Répartir quatre conversions d’images entières entre quatre processus.

programme_2_images_paralleles.py

Les lignes à comprendre

```
with mp.get_context("spawn").Pool(4) as pool:
    mesurer(image, pool)

return pool.map(gris, [image] * LOT, chunksize=1)
```

1. Créer le pool. Pool(4) démarre quatre processus de travail. Le script lance un interpréteur distinct de Spyder pour que les processus puissent démarrer proprement.

2. Distribuer le lot. [image] * LOT fournit quatre tâches à pool.map(). Chaque tâche reçoit une image entière et exécute gris(), qui appelle image.convert('L'). La même photo est répétée pour les mesures.

3. Attendre et enregistrer. map() attend les quatre résultats. Le dernier lot est vérifié pixel par pixel, puis une image est enregistrée dans partie_2_gris.png.

Mesurer sur le Raspberry Pi

Dans Spyder, ouvre le script à côté de **image1.jpg** et appuie sur **F5**. Entre **DÉBUT** et **FIN**, relève la puissance moyenne au wattmètre. La photo est répétée pendant au moins 15 s. Lecture, écriture et démarrage du pool sont hors mesure.

Temps moyen par image (s) = durée / nombre d’images.
Énergie par image (J) = puissance moyenne (W) × temps moyen par image (s).

Puissance moyenne (W)	Temps par image (s)	Énergie par image (J)
À mesurer	À relever dans Spyder	À calculer

Pour la comparaison : Cet essai mesure le débit du traitement d’un lot. Le temps moyen par image ne donne pas le délai d’une image isolée.